

RAY • ANYSCALE

Scaling AI with Ray & Ray Data Pipelines

From a single Python function to distributed training, tuning, batch inference, and online serving — plus production data pipelines.

What is Ray?

- OSS framework for **high-performance, resilient, scale-out** computation on heterogeneous hardware.
- Distributed scheduler for stateless functions (**tasks**) and long-running stateful processes (**actors**).
- Key features:
 - Dependency tracking — **task graphs**
 - Data-movement & resource aware
 - Mixed resources: GPUs, fractional & custom
- Infra: distributed **object store**, fault tolerance via GCS.
- Easy, Python-first APIs and patterns.

Data

Train

Tune

Serve

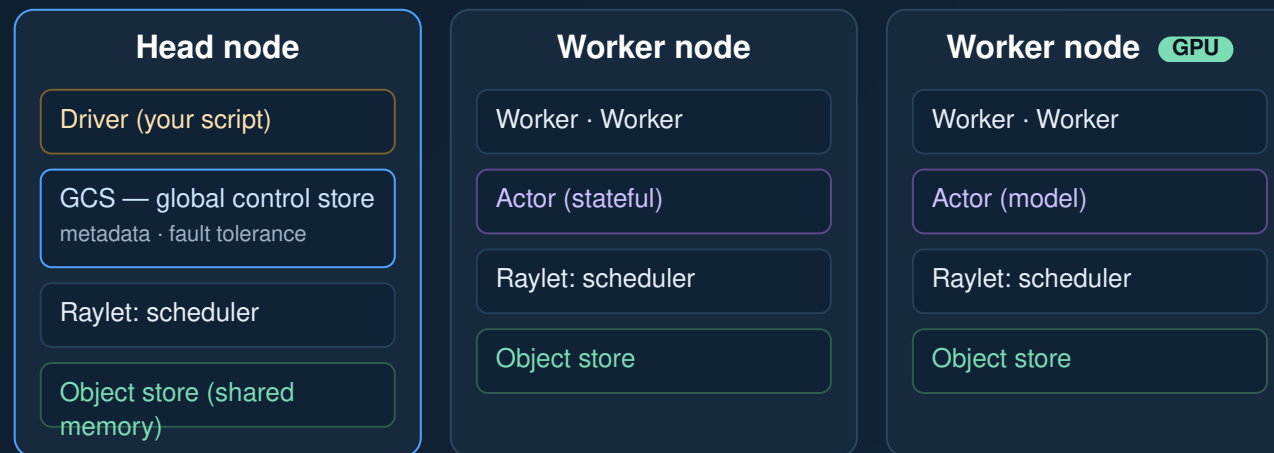
Ray AI Libraries

Ray Core

tasks · actors · objects

CPUs · GPUs · many nodes

ANATOMY OF A RAY CLUSTER



The distributed **object store** spans all nodes — large objects are shared by reference (zero-copy), not copied per task. The head node's **GCS** tracks the whole cluster and enables recovery.

Anyscale: production-ready Ray from day one

Developer central

Multi-node-backed IDE;
advanced observability per
Ray library.

Optimized runtime

Faster performance and
higher GPU utilization vs.
OSS.

Cluster controller

Proactive unhealthy-node
replacement; 0–100 node
~60-second cold starts.

Expertise

Training, 24/7 support, and
professional services.

MANAGED CONTROL PLANE & AUTOSCALING

Anyscale control plane

scheduling · health checks · observability · storage scopes

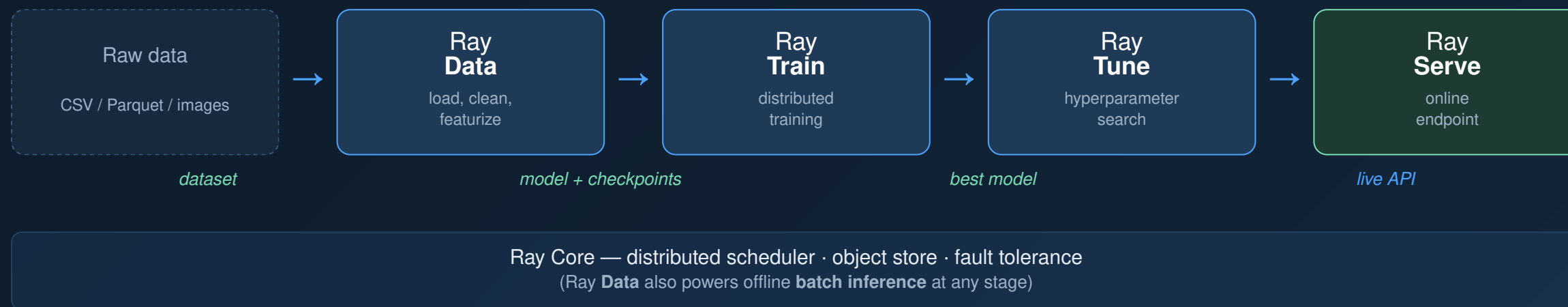
manages & autoscales ↓



The controller adds/removes nodes with demand and swaps unhealthy nodes automatically — you write Ray code; Anyscale runs and heals the cluster.

The Ray AI Libraries — an end-to-end ML workflow

High-level, Python-native APIs for large-scale ML, all built on Ray Core — inheriting its performance, scaling, and fault tolerance. Each library produces an artifact that feeds the next.



A unified framework with an ever-growing ecosystem of integrations — PyTorch, Hugging Face, vLLM, and more — behind a common interface.

End-to-end demo — load & process data with Ray Data

Product-category prediction. Start by reading the catalog and cleaning text. A **per-row** transform uses `map()`.

```
ds = ray.data.read_csv("../ecommerce_product_catalog.csv")

# lowercase, strip punctuation, concat title + description -> "text"
ds_cleaned = ds.map(clean_and_combine)
ds_cleaned.take(5)
```

Datasets are **lazy & streaming**: each operator returns a new Dataset; nothing executes until you take/write the data.

Vectorize & encode with `map_batches`

A **stateful, batch** transform (fitted vectorizer + label map) runs in parallel across block partitions via a class actor.

```
# stateful batch transform -> runs as an actor pool
ds_encoded = ds_cleaned.map_batches(VectorizeAndEncode)

# split for training
train_ds, val_ds = ds_encoded.train_test_split(test_size=0.2, seed=42)
print(train_ds.count(), val_ds.count())
```

```
Train: 800 rows
Val:   200 rows
```

map() = one row at a time | **map_batches()** = a batch + reusable state (e.g. a loaded model).

Train a PyTorch classifier with Ray Train

```
trainer = TorchTrainer(  
    train_loop_per_worker=train_loop,  
    scaling_config=ScalingConfig(num_workers=2),    # scale out  
    datasets={"train": train_ds},  
    run_config=RunConfig(storage_path="/mnt/cluster_storage/"),  
)  
result = trainer.fit()  
result.metrics_dataframe
```

	loss	epoch
0	1.5621	0
1	0.7319	1
2	0.1255	2
3	0.0140	3
4	0.0029	4

- `ScalingConfig` sets the number of distributed workers.
- Ray Train wraps your model in `DistributedDataParallel` and shards the dataset automatically.
- Checkpoints are written each epoch for fault tolerance.

Optimize hyperparameters with Ray Tune

Run many trials in parallel to maximize cluster utilization; swap in smart schedulers/search algorithms to reuse knowledge from completed trials.

```
tuner = Tuner(
    tune.with_parameters(train_driver, dataset=train_ds),
    param_space={"lr": tune.loguniform(1e-4, 1e-1)},
    tune_config=TuneConfig(metric="loss", mode="min", num_samples=3),
)
tune_results = tuner.fit()
best = tune_results.get_best_result("loss", mode="min")
```

	loss	config/lr	trial_id
0	0.002958	0.024665	34c9b_00000
1	0.004143	0.076299	34c9b_00001
2	0.003498	0.004236	34c9b_00002

Batch inference & online serving

Offline — Ray Data

Model is the reusable actor state.

```
class OfflinePredictor:
    def __init__(self):
        self._model = ProductClassifier()
        self._model.load_state_dict(
            torch.load(best_path + "/model.pt"))

    def __call__(self, batch: dict) -> dict:
        with torch.inference_mode():
            o = self._model(
                torch.tensor(batch["features"]))
        return {"prediction": o.numpy()}

preds = val_ds.map_batches(
    OfflinePredictor, concurrency=2)
```

Online — Ray Serve

A class becomes an HTTP deployment.

```
@serve.deployment
class OnlinePredictor:
    def __init__(self, checkpoint):
        self._model = ProductClassifier()
        self._model.load_state_dict(
            torch.load(checkpoint))

    async def __call__(self, request):
        data = json.loads(await request.json())
        return {"prediction": self.predict(data)}

handle = serve.run(
    OnlinePredictor.bind(checkpoint=ckpt))
```

A brief look at Ray Core — tasks

Any Python function you can write, you can distribute. Decorate it, then schedule it with `.remote()`.

```
@ray.remote(num_cpus=2)
def f(a, b):
    return a + b

ref = f.remote(1, 2)    # returns immediately
ref                       # -> ObjectRef(...)

ray.get(ref)            # block & fetch -> 3
```

Python
function

`@ray.remote` ↓

Ray task
(scheduled in cluster)

`.remote()` ↓

ObjectRef
handle / future

ObjectRef = a handle to a (maybe not-yet-computed) result. We don't block until we call `ray.get`.

Task graphs & actors

Task graphs

Pass ObjectRefs as args — Ray resolves dependencies and schedules without blocking.

```
arg1 = add.remote(1, 2)
arg2 = add.remote(10, 20)
out = add.remote(arg1, arg2)    # depends on both
ray.get(out)                   # -> 33
```

Actors — stateful

Long-lived class instances in the cluster. Common use: load a large model **once** and route calls to it.

```
@ray.remote
class Accounting:
    def __init__(self): self.total = 0
    def add(self, amt):  self.total += amt
    def total(self):     return self.total

acc = Accounting.remote()
acc.add.remote(100)
ray.get(acc.total.remote())    # -> 100
```

Ray Data pipeline — ingest & transform

Real workload: prepare an eCommerce product catalog (Parquet). Operators chain into a lazy, streaming pipeline.

```
cat = ray.data.read_parquet("../catalog")           # lazy, streaming
cat.rename_columns(renames) \
    .write_parquet("/mnt/cluster_storage/catalog")   # executes here

# new expression API is legible to the query optimizer:
from ray.data.expressions import col
ds.filter(expr=col("cat") == "Housewares").take(3)  # predicate pushdown
```

Storage scopes on Anyscale — **shared_storage** (org), **cluster_storage** (cluster), per-user & fast local — give best-practice data placement. As of 2026 Ray Data has a query optimizer + planner (logical & physical optimizations).

The core pattern: `map_batches` + an actor

Batch inference (here: text embeddings for semantic search). Constructor holds the heavy state (the model); `__call__` transforms a batch.

```
class AddEmbedding:
    def __init__(self, model_id):
        self.model = SentenceTransformer(model_id)

    def __call__(self, batch, source, dest):
        batch[dest] = self.model.encode(batch[source])
        return batch

ds.map_batches(
    AddEmbedding,
    compute=ray.data.ActorPoolStrategy(size=2),    # pool of model actors
    fn_constructor_args=[model_path],              # -> __init__
    fn_args=["desc", "desc_emb"],                  # -> __call__
)
```

Performance: blocks, parallelism & GPUs

- Ray Data streams data as **blocks** (~100 MB–1 GB), sized automatically — independent of your batch size.
- **One block** → **one actor**. A small dataset = one block = no parallelism.
- `repartition(n)` splits into more blocks → more parallel actors. (1.5 min → much faster.)
- `materialize()` caches blocks in the object store for fast iterative testing.
- GPUs aren't autoscaled — set an explicit `compute` strategy.

```
ray.data.read_parquet(src) \
    .repartition(4) \
    .map_batches(
        AddEmbedding,
        num_gpus=1,           # schedule on GPU
        batch_size=64,        # fit GPU RAM
        compute=ray.data.ActorPoolStrategy(size=2),
        fn_constructor_args=[model_path],
        fn_args=["desc", "desc_emb"],
    ).write_parquet(dest)
# GPU run: ~10s (vs ~90s CPU, 1 actor)
```

Tip: if a pipeline is interrupted, stray actors may hold resources — `ray.shutdown()` resets cleanly.

Scaling LLM inference with RayLLM + vLLM

Generate multimodal catalog copy (image + text → text). Same `map_batches` idea, but a managed vLLM engine replaces the hand-written actor — faster and simpler.

```
config = vLLMEngineProcessorConfig(
    model_source=model_path,
    engine_kwargs=dict(max_model_len=8192,
                      limit_mm_per_prompt={"image": 1}),
    batch_size=8, concurrency=2,          # ~= GPUs used
)
processor = build_llm_processor(
    config,
    preprocess=lambda row: dict(messages=[...], sampling_params=...),
    postprocess=lambda row: dict(cat_desc=row["generated_text"], ...),
)
ds = processor(ray.data.read_parquet(src))
```

Takeaway: one mental model — tasks, actors, and `map_batches` — scales from a toy function to distributed training, tuning, batch & online inference, and production LLM pipelines. AI assistants can even write much of the Ray code for you.